

Cybersecurity Lab Report

Phases 1 & 2 – Virtual Lab Environment Setup & Multi-VM Networking

Kali Linux & Windows 10 on VirtualBox with a Custom NAT Network

1. Objective

Set up a virtual lab environment for cybersecurity practice by installing 7-Zip and Oracle VirtualBox, creating a custom NAT Network on the 10.0.0.0/24 subnet, importing a prebuilt Kali Linux virtual machine, configuring its IP settings to operate on that subnet, and capturing a snapshot of the completed state.

2. Tools & Resources Used

Archive Utility	7-Zip 26.02 (x64)
Hypervisor	Oracle VirtualBox 7.2.16
Guest OS	Kali Linux 2026.2 (prebuilt VirtualBox image, amd64)
Target Network	10.0.0.0/24 (custom NAT Network)
Host OS	Windows (Microsoft Surface device)

3. Step-by-Step Process

3.1 Install 7-Zip

7-Zip was downloaded from 7-zip.org and installed to the default location (C:\Program Files\7-Zip\). This tool is required later to extract the compressed Kali Linux VM archive.

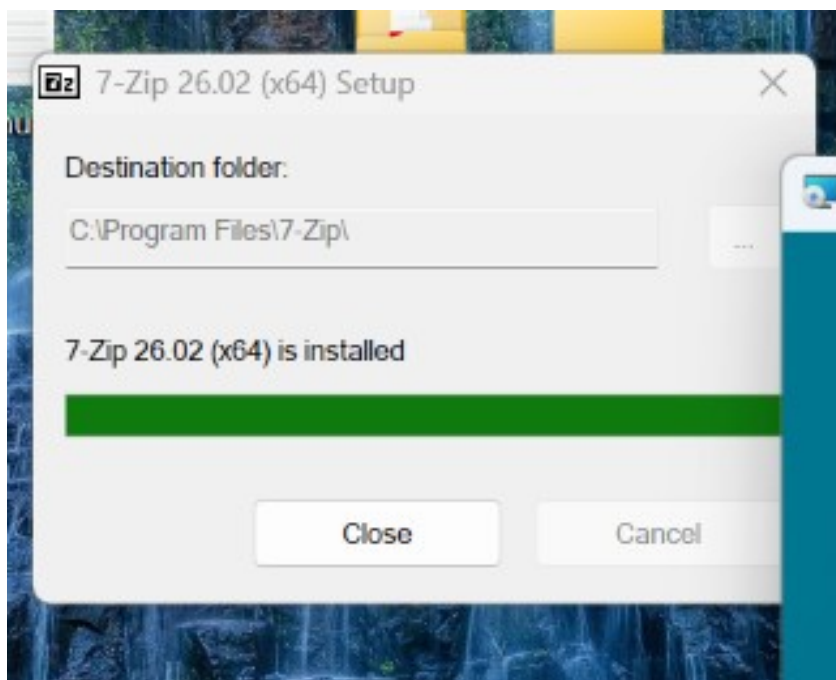


Figure 1 — 7-Zip 26.02 (x64) installation completed.

3.2 Install Oracle VirtualBox

Oracle VirtualBox 7.2.16 was downloaded from [virtualbox.org](https://www.virtualbox.org) and installed using the default installation options.

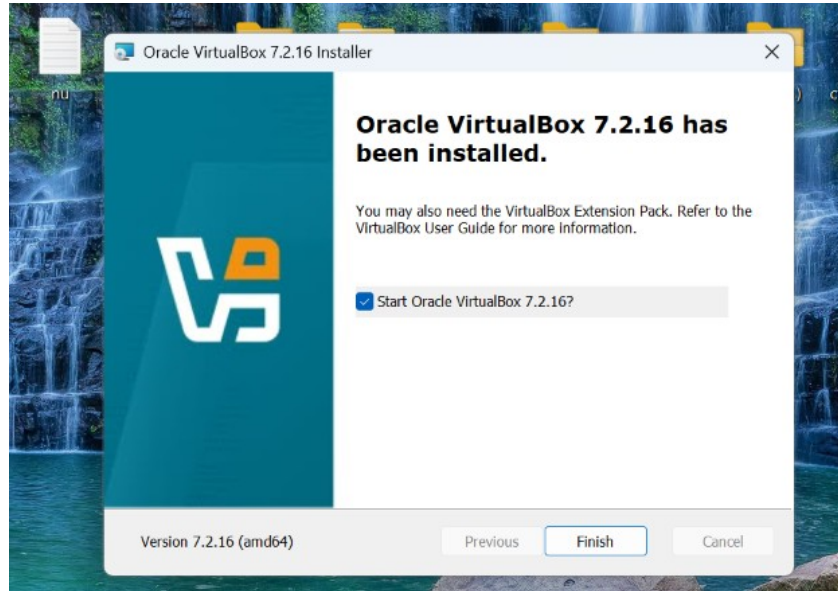


Figure 2 — Oracle VirtualBox 7.2.16 installation completed.

3.3 Configure the NAT Network

A NAT Network named "NatNetwork1" was created via File > Tools > Network Manager > NAT Networks in VirtualBox. The lab required this network to use the 10.0.0.0/24 subnet.

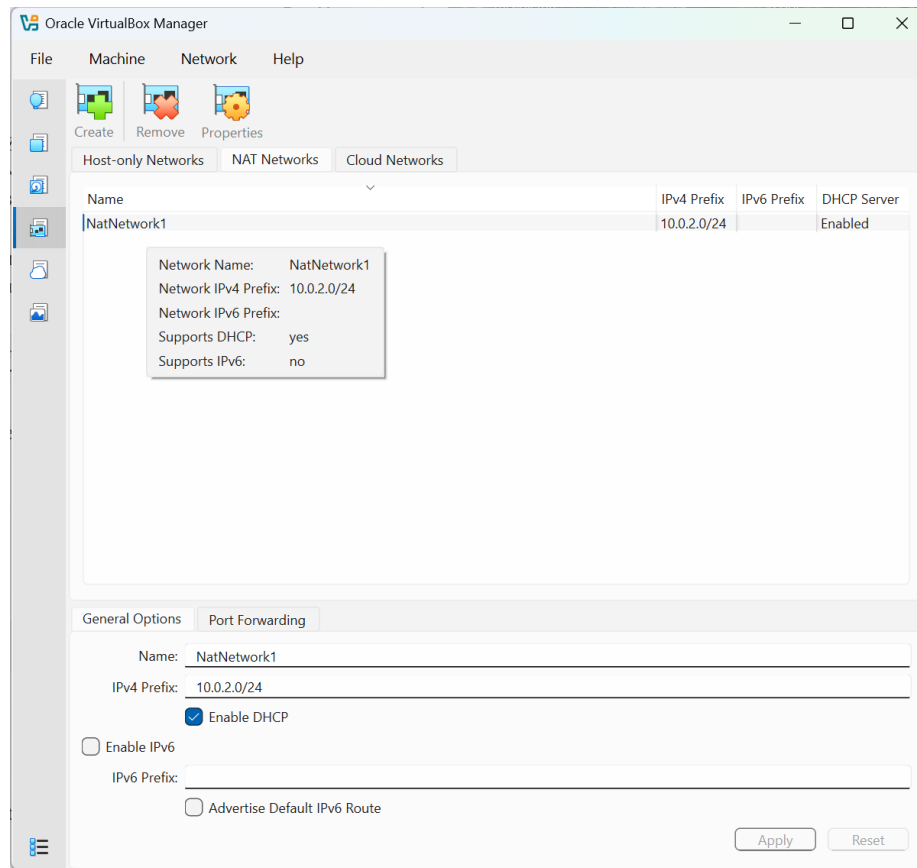


Figure 3 — NatNetwork1 initially created with the default IPv4 prefix of 10.0.2.0/24, instead of the required 10.0.0.0/24.

3.4 Import and Attach the Kali Linux VM

The prebuilt Kali Linux 2026.2 VirtualBox image was downloaded from kali.org/get-kali, extracted using 7-Zip, and imported into VirtualBox via File > Import Appliance. The VM's network adapter (Adapter 1) was then set to "Attached to: NAT Network" and pointed at NatNetwork1.

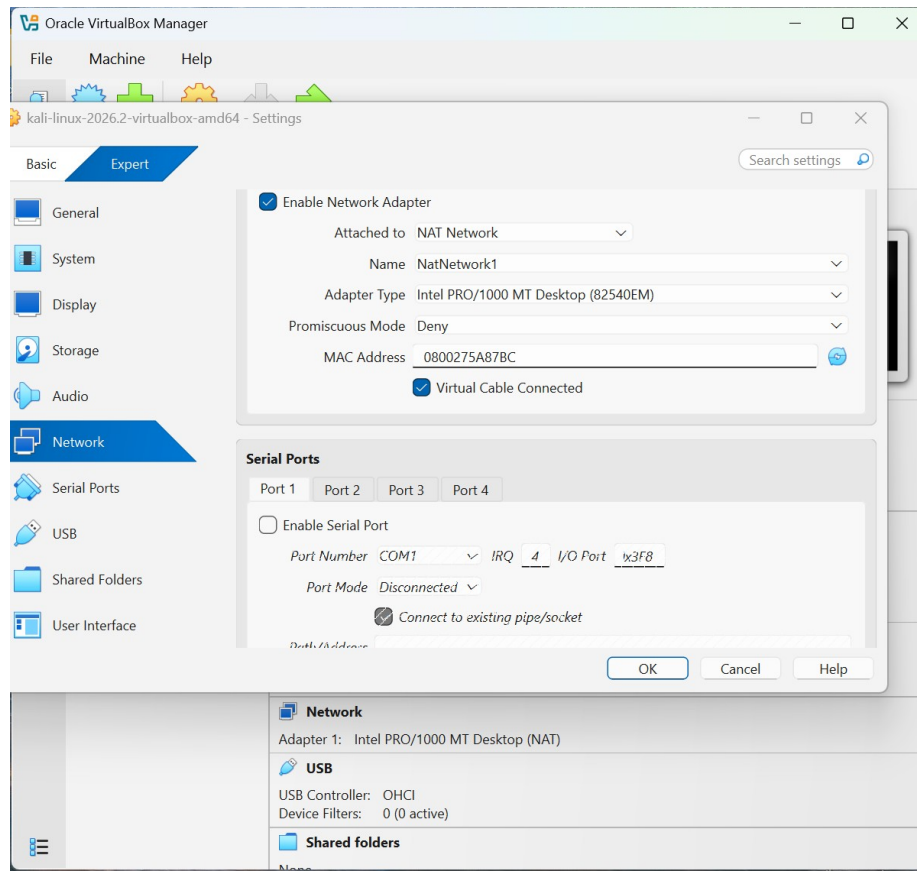


Figure 4 — Kali Linux VM network adapter configured to attach to NatNetwork1.

4. Issue Encountered & Troubleshooting

After booting the VM and running "ip a", the eth0 interface received the address 10.0.2.15/24 — the VirtualBox default NAT range — rather than an address in the required 10.0.0.0/24 subnet. This indicated the NatNetwork1 network was still using its default IPv4 prefix rather than the value the lab specified.

- Re-opened Network Manager and confirmed NatNetwork1 was still set to 10.0.2.0/24 (see Figure 3).
- Edited the IPv4 Prefix field to 10.0.0.0/24 and clicked Apply.
- Fully powered off the Kali VM and restarted it to force a new DHCP request, but eth0 still returned 10.0.2.15/24 — confirming the issue was in VirtualBox's NAT engine, not the guest OS.
- Attempted a DHCP lease renewal inside Kali using nmcli (the correct tool for this Kali release, since dhclient is no longer included) — this also had no effect, ruling out a guest-side caching issue.
- Restarted VirtualBox Manager entirely to clear any cached NAT Network state.
- Re-verified the NatNetwork1 configuration and confirmed the IPv4 Prefix now correctly displayed 10.0.0.0/24.

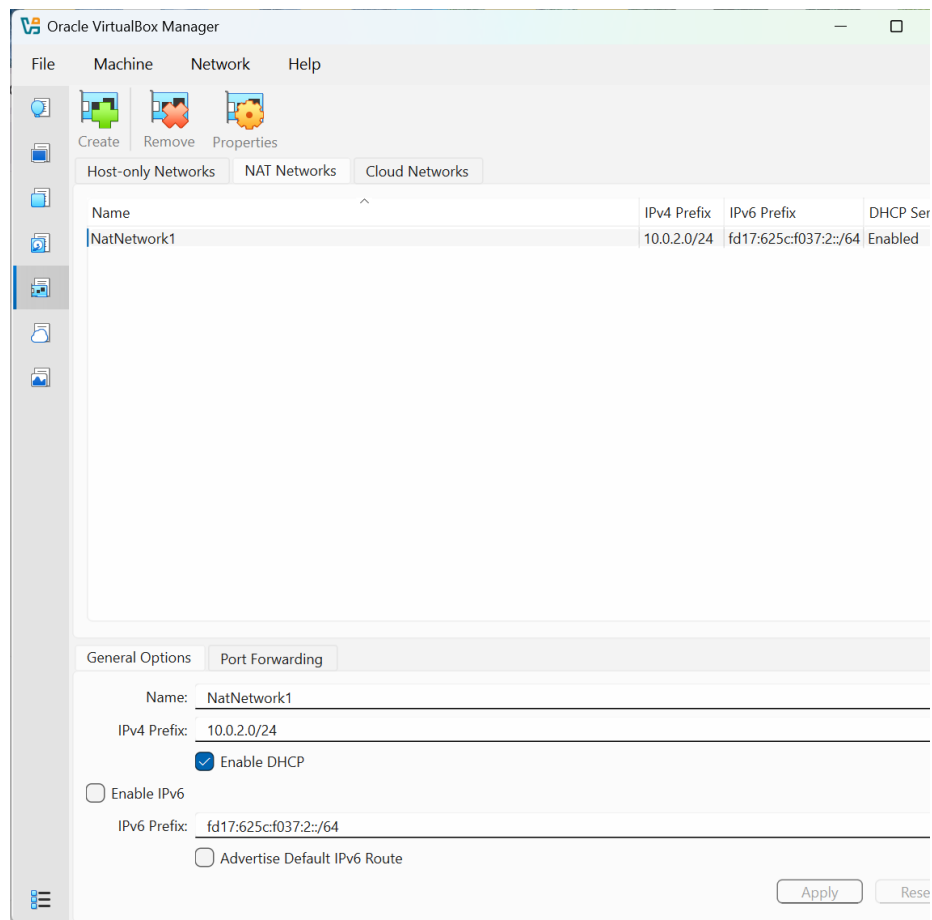


Figure 5 — NatNetwork1 configuration screen while diagnosing the incorrect subnet.

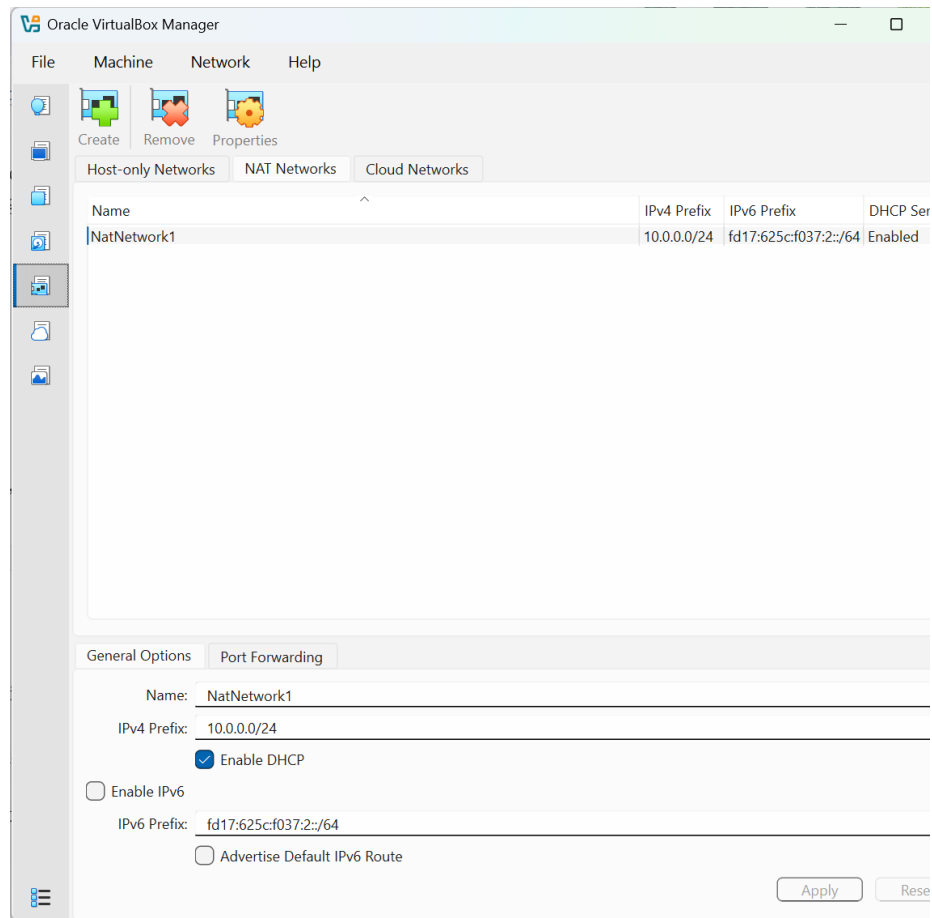
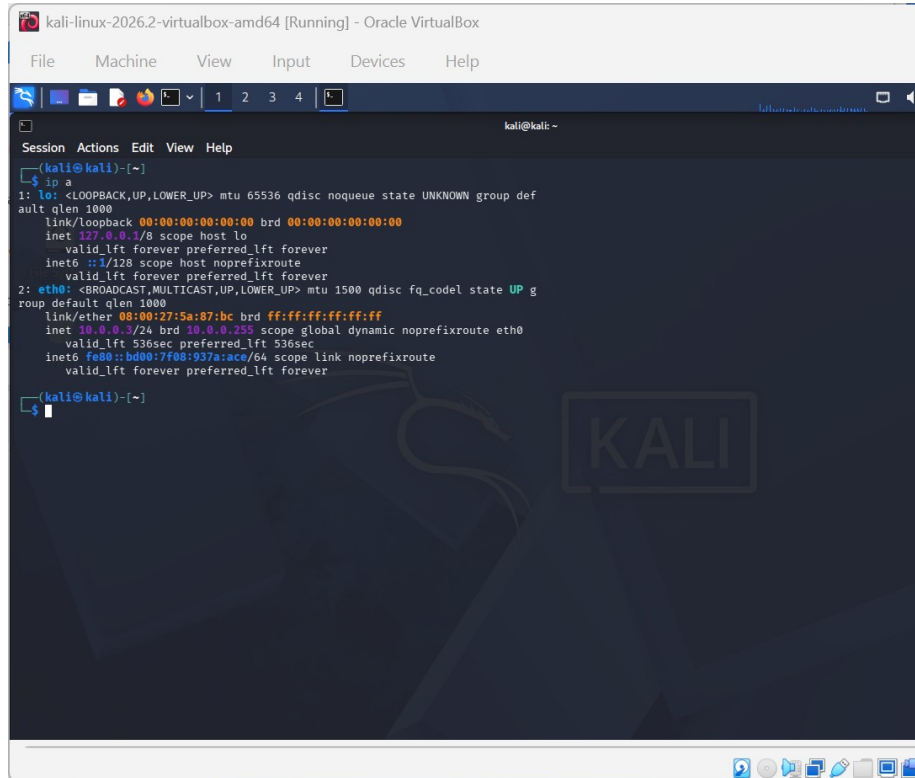


Figure 6 — NatNetwork1 successfully updated to the required IPv4 Prefix of 10.0.0.0/24.

Once the corrected NAT Network configuration was confirmed and the VM restarted, the Kali Linux VM obtained an address within the correct subnet.

5. Verified Network Configuration



```
kali@kali:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
    ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
    roup default qlen 1000
    link/ether 00:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.3/24 brd 10.0.0.255 scope global dynamic noprefixroute eth0
        valid_lft 536sec preferred_lft 536sec
    inet6 fe80::bd00:7f00:937a:ace/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Figure 7 — "ip a" output confirming eth0 received 10.0.0.3/24, within the required 10.0.0.0/24 subnet.

Interface	eth0
IPv4 Address	10.0.0.3/24
Broadcast	10.0.0.255
Assignment Method	Dynamic (DHCP via NatNetwork1)
Result	Successfully placed on the required 10.0.0.0/24 subnet

6. Conclusion

Phase 1 was completed successfully. All required software (7-Zip, VirtualBox) was installed, a custom NAT Network was created and corrected to the specified 10.0.0.0/24 subnet, the Kali Linux VM was imported and attached to that network, and its IP configuration was verified as 10.0.0.3/24. A snapshot of this completed state was taken to preserve a clean rollback point before beginning further lab exercises.

The main challenge in this phase was a NAT Network subnet mismatch, where the network defaulted to 10.0.2.0/24 instead of the required 10.0.0.0/24. This was resolved by correcting the IPv4 Prefix in VirtualBox's Network Manager, fully restarting both the VM and VirtualBox Manager to clear cached network state, and confirming the fix by re-checking the interface configuration inside the guest OS.

Phase 2 – Multi-VM Networking & Ping Testing

7. Objective

Add a second virtual machine (Windows 10) to the lab environment, join it to the same NatNetwork1 (10.0.0.0/24) used by the existing Kali Linux VM, verify two-way network connectivity between both machines using ping, and take snapshots of both VMs once connectivity was confirmed.

8. Step-by-Step Process

8.1 Install Windows 10

A Windows 10 ISO was used to create a new VM in VirtualBox and installation was completed through the standard Windows Setup wizard.

8.2 Initial Network Check

After installation, the Windows VM's network adapter was still attached to VirtualBox's default "NAT" mode rather than "NAT Network". Running ipconfig showed an IPv4 address of 10.0.2.15 — the VirtualBox default NAT range — confirming it was not yet on the same subnet as the Kali VM.

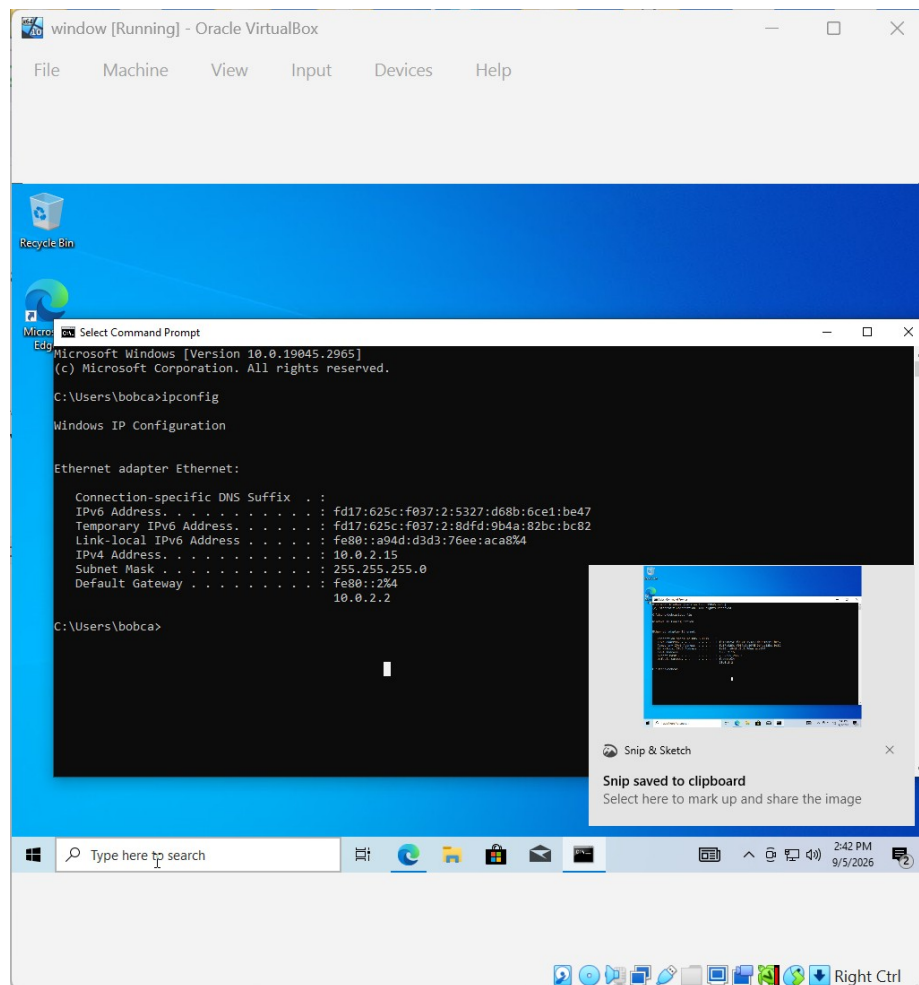


Figure 8 — Windows 10 initially receiving 10.0.2.15, the default VirtualBox NAT range, before being attached to NatNetwork1.

8.3 Attach Windows to NatNetwork1

The Windows VM was shut down, and in Settings > Network > Adapter 1, "Attached to" was changed from NAT to NAT Network, with NatNetwork1 selected. After restarting the VM, ipconfig confirmed a corrected address of 10.0.0.4/24 — on the same subnet as Kali (10.0.0.3/24).

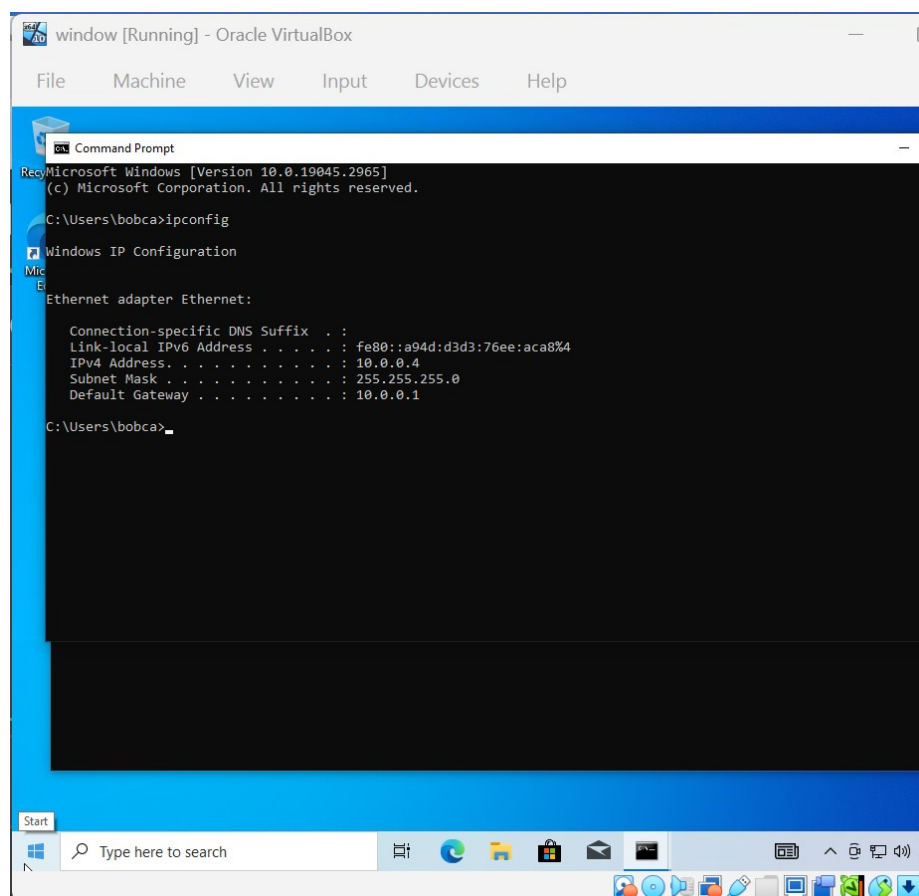


Figure 9 — Windows 10 correctly assigned 10.0.0.4/24 after attaching to NatNetwork1.

9. Ping Testing

9.1 Windows → Kali

With the Kali VM running and confirmed on 10.0.0.3, a ping was issued from Windows to Kali: "ping 10.0.0.3". This succeeded with 4 of 4 packets received and 0% loss, confirming outbound connectivity from Windows to Kali.

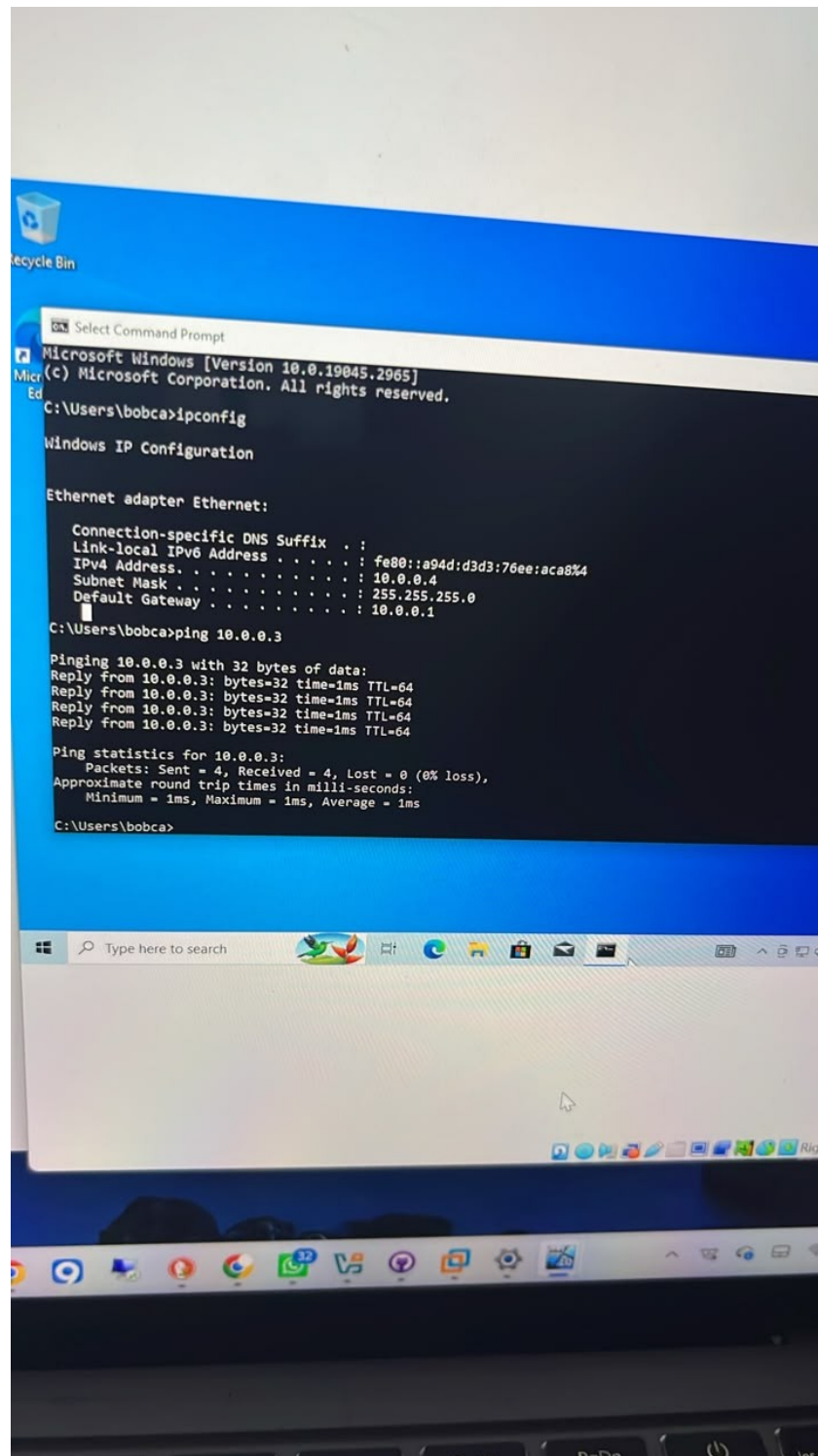


Figure 10 — Successful ping from Windows (10.0.0.4) to Kali (10.0.0.3): 4 replies, 0% packet loss.

9.2 Kali → Windows (initial attempt)

A reverse ping was then run from Kali to Windows: "ping -c 4 10.0.0.4". This initially resulted in 100% packet loss. This is expected default behavior — Windows Defender Firewall blocks inbound ICMP Echo Request traffic by default, even though outbound pings and replies to pings it initiates are permitted.

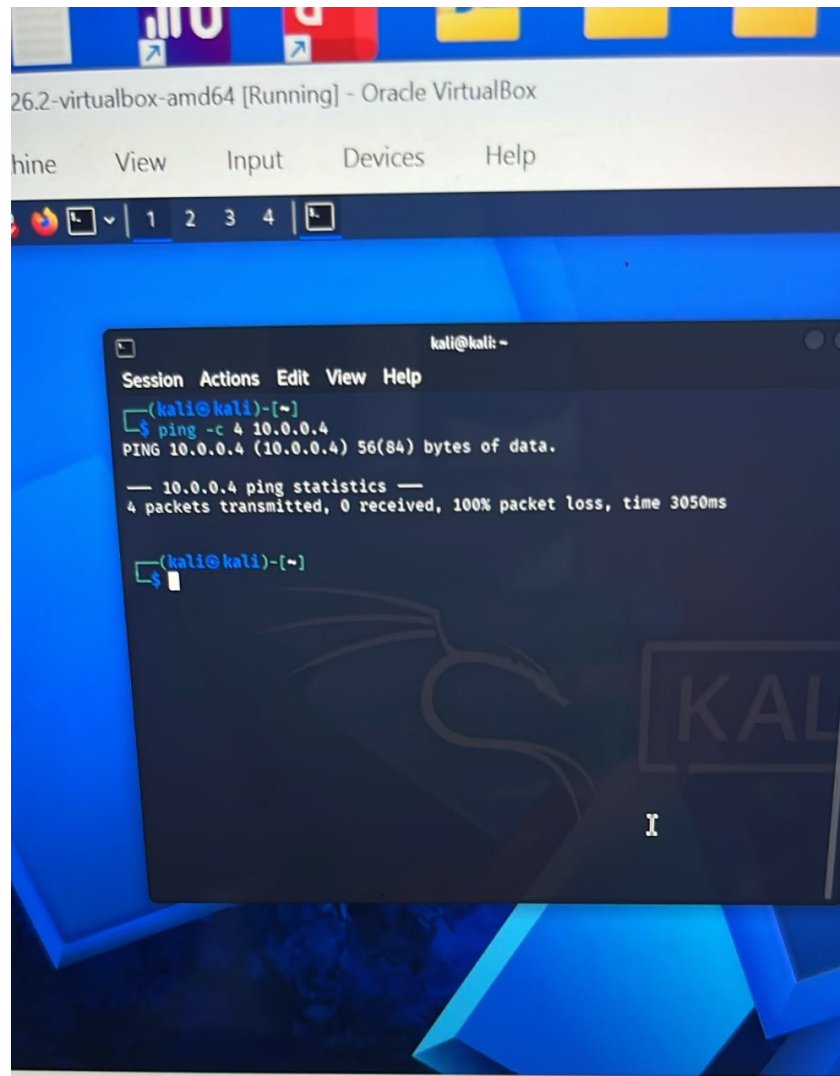


Figure 11 — Initial ping from Kali to Windows failing with 100% packet loss due to Windows Firewall blocking inbound ICMP.

9.3 Enabling Inbound ICMP on Windows

To resolve this, Windows Defender Firewall with Advanced Security was opened, and under Inbound Rules, the "File and Printer Sharing (Echo Request - ICMPv4-In)" rules were located and enabled for the relevant network profiles, allowing incoming ping requests to be answered.

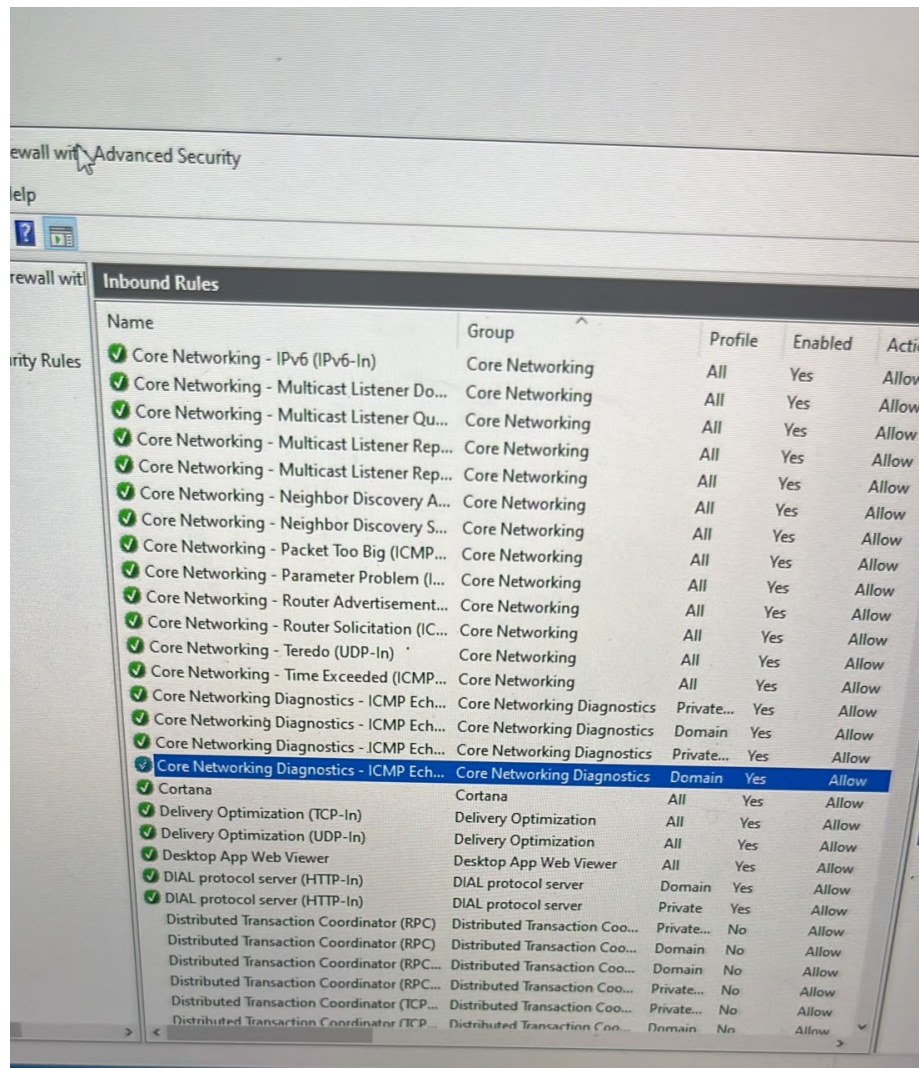
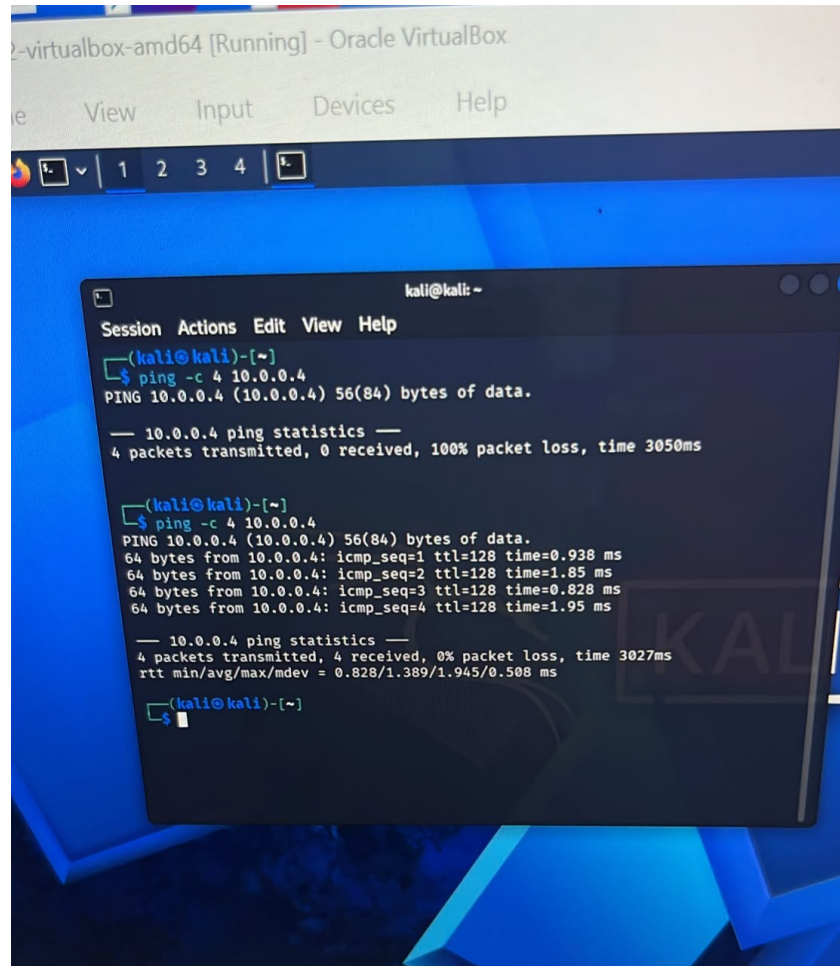


Figure 12 — Windows Defender Firewall Inbound Rules showing ICMP Echo Request rules enabled.

9.4 Kali → Windows (after firewall fix)

The ping from Kali to Windows was re-run ("ping -c 4 10.0.0.4") and this time succeeded with 4 of 4 packets received and 0% packet loss, confirming full two-way connectivity between both virtual machines on NatNetwork1.



```
kali@kali: ~  
Session Actions Edit View Help  
$ ping -c 4 10.0.0.4  
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data:  
  
— 10.0.0.4 ping statistics —  
4 packets transmitted, 0 received, 100% packet loss, time 3050ms  
  
$ ping -c 4 10.0.0.4  
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data:  
64 bytes from 10.0.0.4: icmp_seq=1 ttl=128 time=0.938 ms  
64 bytes from 10.0.0.4: icmp_seq=2 ttl=128 time=1.85 ms  
64 bytes from 10.0.0.4: icmp_seq=3 ttl=128 time=0.828 ms  
64 bytes from 10.0.0.4: icmp_seq=4 ttl=128 time=1.95 ms  
  
— 10.0.0.4 ping statistics —  
4 packets transmitted, 4 received, 0% packet loss, time 3027ms  
rtt min/avg/max/mdev = 0.828/1.389/1.945/0.508 ms  
$
```

Figure 13 — Successful ping from Kali (10.0.0.3) to Windows (10.0.0.4) after enabling inbound ICMP: 4 replies, 0% packet loss.

10. Snapshots

With both machines confirmed on the 10.0.0.0/24 subnet and two-way ping connectivity verified, both VMs were shown running simultaneously in VirtualBox Manager, with the Kali VM's Network details confirming attachment to NatNetwork1. Snapshots were then taken of both the Windows 10 VM and the Kali Linux VM to preserve this verified, working state as a rollback point.

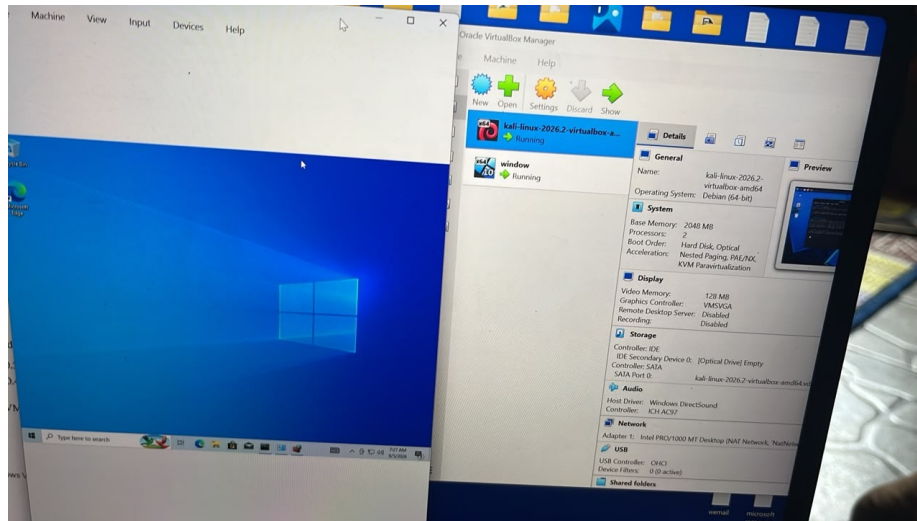


Figure 14 — Both the Kali Linux and Windows 10 VMs running simultaneously in VirtualBox Manager, with Kali's Network details confirming attachment to NatNetwork1.

Kali Linux IP	10.0.0.3/24
Windows 10 IP	10.0.0.4/24
Shared Network	NatNetwork1 (10.0.0.0/24)
Windows → Kali Ping	Success (0% loss)
Kali → Windows Ping	Success after enabling inbound ICMPv4 (0% loss)
Snapshots Taken	Windows 10 VM and Kali Linux VM

11. Conclusion

Phase 2 was completed successfully. A Windows 10 VM was installed and joined to the same NatNetwork1 subnet as the existing Kali Linux VM. Two-way ping connectivity was verified between both machines, requiring an inbound ICMPv4 firewall rule change on Windows to allow it to respond to ping requests from Kali. Snapshots of both VMs were taken once connectivity was confirmed, providing a clean rollback point before further lab work.